Problem 7
Joseph Coombs, Abdul Rehman, Muhammad Abdullah
COM 316 Artificial Intelligence

While facts are crucial in helping our robot find the goal, these facts might not always accurately reflect the environment the robot is in, especially if said environment changes frequently or if the goal moves. To help address this, we store in memory the facts obtained from the previous traversals so we can make changes to them over time.

Our process to do this (outlined below) consists of several different steps. First, we start with our current node and look at all the neighbors. Then, we check if the current position is equal to our goal. If so, we stop and return a success message. Next, we need to clean up memory from previous traversals. So, if any of our current neighbors have already been visited (i.e. they are in the list old_locations) and that location is not our current position, then we mark that neighbor as an obstacle and delete the fact that the old visited location and current node are adjacent. Then, we can check for obstacles around us. If any of the neighbors is an obstacle then we also delete the fact that the neighbor is adjacent to the current node. We could also check if any neighbor has already been visited, and if so, delete the fact that the neighbor is adjacent to our current node.

Next, we check if any neighbor is left and has a free status (or not an obstacle status). If so, we visit it, add it to the memory, and save the fact that the current node is the parent of that neighbor. Then, we update the current position to that neighbor and move there. This way, we have checked all possible neighbors and eliminated the locations where we don't want to go. We are only left with free nodes and we choose them.

If we are not left with any free neighbors, this means we are at a dead end and we set the current location to the parent node and move there and the rules above would mark the dead end node as an obstacle.

There are a few more cases our rules need to account for. If the robot backtracks all the way back to the very first node it started at and that node also has zero valid neighbors left, it means no path to the goal exists at all, so we need one more rule that checks if num_valid_neighbors(current) = 0 and parent(current) does not exist, and if so, stops and returns a “Failure” message instead of just stalling with no result. We also want our backtrack rule to actually mark the dead end it’s leaving behind as an obstacle (add_to_memory(obstacle_status(current)) before updating position), so that on a future traversal the robot remembers not to walk back into that same dead end and waste time re-exploring it.

Lastly, since the environment isn’t known in advance, we rely on the given function that senses the neighbors of whatever node the robot is currently standing on to generate our adjacent_path and obstacle_status facts, so this sensing call needs to happen automatically right after every move (whether moving forward or backtracking) so the current node’s neighbor facts exist before any of the other rules try to check them.

```
If equal(position(current) position(goal))
then
stop and return “Success”

If adjacent_path(old_location, neighbor) exists and not
equal(old_location, position(current))
Then
delete(adjacent_path(old_location, neighbor))
delete(obstacle_status(neighbor))

If adjacent_path(position(current), neighbor) exists and equal
(status(neighbor),obstacle)
Then
delete(adjacent_path(position(current) neighbor))

If adjacent_path(position(current), neighbor) exists and
has_been_visited(neighbor) then
delete(adjacent_path(position(current), neighbor))

If adjacent_path(position(current), neighbor) exists and
equal(status(neighbor), free) then add_to_memory(visited(neighbor))
add_to_memory(parent(current_is_parent_of_neighbor))
delete(position(current)) set(position(current) = neighbor)
execute(move_forward_to(neighbor))

If num_valid_neighbors(current) = 0 and parent(current) exists
Then
set(position(current) = position(parent_node)
execute(move_to(parent_node))
```